Cross-Lingual Supervision improves Large Language Models Pre-training

Andrea Schioppa
Google Research
arischioppa@google.com

Xavier Garcia
Google Research
xgarcia@google.com

Orhan Firat
Google Research
orhanf@google.com

arXiv:2305.11778v1 [cs.CL] 19 May 2023

Abstract

The recent rapid progress in pre-training Large Language Models has relied on using self-supervised language modeling objectives like next token prediction or span corruption. On the other hand, Machine Translation Systems are mostly trained using cross-lingual supervision that requires aligned data between source and target languages. We demonstrate that *pre-training* Large Language Models on a mixture of a self-supervised Language Modeling objective and the supervised Machine Translation objective, therefore *including cross-lingual parallel data during pre-training*, yields models with better in-context learning abilities. As pre-training is a very resource-intensive process and a grid search on the best mixing ratio between the two objectives *is prohibitively expensive*, we propose a simple yet effective strategy to learn it during pre-training.

1 Introduction

The rapid progress in the development of large-scale pre-training, GPT (Brown et al., 2020), XGLM (Lin et al., 2021), PaLM (Chowdhery et al., 2022), has resulted in models capable of performing a variety of tasks through the *in-context learning (aka. few shot) paradigm* (Brown et al., 2020): one can present the model a few demonstrations of a given task at inference, and the model will able to follow these demonstrations on *new, unseen examples*. Therefore it is *no longer necessary to fine-tune* these models on a variety of down-stream tasks. The pre-training of such Large Language Models (LLMs) *relies on self-supervision*, i.e. the data does not require to be annotated. Examples of self-supervised (LM) Language Modeling objectives are next token prediction, where the task is to predict the next token given the previous ones, or span-corruption where the task is to fill-in a portion of missing text given its surroundings.

On the other hand, Machine Translation Models (MTMs) are still being trained using cross-lingual supervision, which *requires aligned parallel data*. Indeed, the Machine Translation (MT) objective consists in predicting the target sentence given the source sentence, and therefore it is *necessary to collect aligned pairs* of texts between source and target languages.

On Machine Translation, pre-trained LLMs have historically under-performed MTMs trained just on millions of supervised examples both when the LLMs are *evaluated using in-context learning*, or *after having been fine-tuned on parallel data*. However, the performance gap between LLMs and MTMs has been shrinking. For example, the recent PaLM (Chowdhery et al., 2022), a language model pre-trained using self-supervision only, is able to outperform previous state-of-the-art MTMs on older machine translation benchmarks, while still lagging behind supervised MTMs on recent benchmarks (Vilar et al., 2022). Such a trend raises the natural question Q: *Is training on cross-lingual supervised data still necessary or beneficial?*

Regarding question Q, we think that the most promising direction to explore is the *inclusion of parallel data when pre-training LLMs*. The first rationale for our preference is the shrinking gap on MT benchmarks between LLMs and MTMs: it is quite likely that LLMs will be able to catch-up in the nearby future, while at the same time being able to perform many more tasks than MTMs. The second rationale is that pre-training datasets are still dominated by English, compare the language composition of the pre-training dataset for PaLM (Chowdhery et al., 2022): other languages, especially lower resource ones, *are under-represented*. Therefore, a natural conjecture is that aligned cross-lingual data *might enhance the abilities of LLMs across languages other than English*.

When assessing the multi-lingual abilities of LLMs we need to distinguish between the *open and closed generation* settings. In closed generation the

task *is performed in a single language*; for example a context paragraph is presented in German, questions are formulated in German and answers are expected in German. In open generation the task *is performed across two languages*; for example a context paragraph is presented in English, questions are formulated in German and answers are expected in German. Now the attractiveness of including cross-lingual data during pre-training lies not only in the ability to improve the machine translation performance of LLMs, but also in *building a bridge between languages*. While we might expect that cross-lingual supervision improves closed generation in under-represented languages, another natural conjecture is that it improves open generation, i.e. where two languages are involved.

In light of the above discussion we refine Q: *Is cross-lingual supervised data beneficial when pre-training LLMs? In particular, are there gains both on open and closed generation when using the in-context learning paradigm for evaluation?*

We are not the first to consider the usage of cross-lingual supervision with LLMs, see Section 2. However our study differs from previous ones in the following aspects:

1. We include cross-lingual supervision at the *pre-training stage*.

2. We include cross-lingual supervision using the standard supervised MT objective.

3. We evaluate the resulting models with in-context learning considering *both closed and open generation settings*.

4. We *learn* the amount of parallel data to use *while training*.

In this work we first demonstrate that including *some cross-lingual supervision is beneficial when pre-training large language models*, thus answering Q. Then, when faced with learning an optimal amount of cross-lingual supervision to use, we show that automated curriculum learning (Graves et al., 2017) is *an effective strategy that does not require multiple training runs and which outperforms static policies*.

We emphasize the importance of learning the amount of parallel data while training *without resorting to a hyper-parameter search*. Pre-training an LLM on sufficiently many tokens is a *resource intensive task*; for example each of our experiments with a 3.8B-parameter models requires 256 TPUv4 cores for 5 days. If we treat the mixing ratio between the parallel MT data and the LM training data as a hyper-parameter λ, we have *in theory just an additional hyper-parameter*. However a grid search is *prohibitively expensive*; for example (Kale et al., 2021) considered a *less compute-intensive* setup in which one *fine-tunes mT5 models for 100k steps* on a mixture of MT and LM data; nevertheless they were able to just compare *two values of* λ. Furthermore, treating λ as a hyper-parameter overlooks the fact that there might be *dynamic scheduling strategies*, i.e. varying λ over time, that outperform static ones in which λ is held fixed.

2 Related work

We are not the first to investigate the usage of parallel cross-lingual data with LLMs. (Reid and Artetxe, 2022) considered leveraging parallel data by devising a loss consisting of 3 objectives; however, their technique is somewhat complicated because it necessitates the development of a multilingual noising procedure, while we opt for including the cross-lingual data using the standard MT objective. (Chi et al., 2021) proposed a simpler objective by building on top of the success of (Xue et al., 2021): directly adding supervised MT data to the denoising procedure used for training mT5, which results in models outperforming mT5 in cross-lingual generation. Note however, that while (Chi et al., 2021) includes cross-lingual supervision during pre-training, the resulting models *do not display in-context learning abilities* and evaluation is carried out by fine-tuning on down-stream tasks. (Kale et al., 2021) explored what happens by fine-tuning mT5 on parallel data; therefore parallel data is used during an *intermediate stage between pre-training and fine-tuning on down-stream tasks*. A limitation of all these studies is the emphasis on *fine-tuning*: all of these models require fine-tuning, which is quite different from few-shot in-context learning. As such, the question of whether supervised data in one task can benefit few-shot learning in another task *remains unexplored*.

3 Basic Setup

3.1 Training Data

Our Language Modeling data is based on that from (Chowdhery et al., 2022) but we slightly

Data Source.	% of Data
Social media conversations†	40%
Filtered webpages†	34%
GitHub	4%
Books*	15%
Wikipedia†	5%
News*	2%

Table 1: LM Data: Data sources and proportion of data. † means the data source is multilingual, while ∗ means it is English-only.

modify the proportions between different subcategories, see Table 1. We do not use a public Language Modeling dataset, e.g. MC4 (Raffel et al., 2019), as in early experiments the high-quality data from (Chowdhery et al., 2022) yielded better in-context learning abilities. As Language Modeling objective we use the recent "UL2" (Tay et al., 2022) because it has shown better performance in the few-shot setting.

For the MT data we use an in-house parallel corpus covering the languages in Table 2, which also reports the sampling proportions and highlights whether we consider a language in the High or Low resource setting. Note that our training data has always the source or the target in English. We use the standard approach used when training multi-lingual supervised models:

$$\langle 2xx \rangle + \text{source} \rightarrow \text{encoder} \quad (1)$$

$$\text{target} \rightarrow \text{decoder}, \quad (2)$$

where the source sentence is prefixed with a special target language token, $\langle 2xx \rangle$, and is supplied to the Encoder, while the target is supplied to the Decoder.

3.2 Model architecture

Commonly used LLM architectures are Encoder-Decoder models, e.g. T5 (Raffel et al., 2019), and Decoder-only models, e.g. (Brown et al., 2020; Chowdhery et al., 2022). Most supervised MTMs use an Encoder-Decoder architecture. As our experiments require *pre-training from scratch* and are therefore quite resource-demanding, we *consider only one architecture, the Encoder-Decoder*. Specifically, we use the mT5 (Xue et al., 2021) architecture at model sizes "large" (1.2 billion) and "xl" (3.8 billion). We train the 1.2B models for 250k steps and the 3.8B billion models for 500k steps using the default settings from the T5X library (Roberts et al., 2022). In our batches the maximum sequence length is 1024 and the number of non-padding tokens is slightly over 500k. We emphasize that *mT5 is only used for the architecture, and we never use mT5 checkpoints or the data used to train mT5*.

Data Source.	% of Data	Low/High	Tokens (B)
Sentences	96%	-	-
Documents	4%	-	-
ar	7.3%	High	16.5
bn	5.4%	Low	1.4
de	9.5%	High	85.7
fi	6.3%	High	8.2
fr	9.8%	High	123.6
id	7.1%	High	5.8
ja	7.9%	High	27.9
ko	7.2%	High	15.8
ru	8.6%	High	54.6
sw	4.8%	Low	1.1
te	4.5%	Low	0.5
th	6.6%	High	14.3
tr	7.9%	High	11.9
vi	7.1%	High	12.7

Table 2: MT Data composition

3.3 Evaluation

We evaluate our models with in-context learning using the one-shot setting, see the Appendix for explicit examples; concretely, each test input is prefixed with one example displaying the desired input to target behavior; the sequence thus obtained is supplied to the Encoder and the target is generated by the Decoder.

We consider three tasks: Question Answering, Machine Translation and Summarization. For Question Answering we consider two settings: *closed generation*, where the context, the question and the answer are in the same language, and *open generation* where the context is in one language and the question and the answer are in another one. For the closed generation setting we use TyDiQA (Clark et al., 2020). For the open generation setting we take the non-English splits of TyDiQA and translate the context to English using the Google Translate API (`translate.google.com` accessed in November 2022.); we denote the dataset thus obtained as XTyDiQA. For Machine Translation we use Flores (Guzmán et al., 2019) and for Summarization we employ Wikilingua (Ladhak et al., 2020), with the splits and preprocessing from the GEM (Gehrmann et al., 2022) benchmark.

4 Learning to schedule the two tasks

A grid search on λ is unfeasible. As we have two tasks, Language Modeling and Machine Translation, we might treat the proportion λ of the MT task as a hyper-parameter to tune. Given that pre-training is very resource intensive, a *grid search on λ is unfeasible*. Even in the less compute intensive setting considered by (Kale et al., 2021), which is a continued pre-training of mT5 checkpoints, they were able to compare *just two values of* λ. It is therefore highly desirable to learn λ during training, with the additional benefit that a policy changing λ over time might outperform one that holds it constant.

Automated curriculum learning is a natural approach. When training a model on data from multiple sources, the *automated curriculum learning paradigm* (Graves et al., 2017) can learn the data-sampling schedule while training. In this way we can learn a dynamic lambda, λ_t, which is a function of the time step t; concretely, λ_t represents the probability of sampling the MT task and $1 - \lambda_t$ is the probability of sampling the LM task. Recent work (Kreutzer et al., 2021) has shown promising results when applying this curriculum approach to Machine Translation Systems where the data comes from multiple domains or multiple languages. For example, (Kreutzer et al., 2021) demonstrates that the multi-armed bandits employed by automated curriculum learning perform competitively against several SOTA heuristics on multi-lingual benchmarks.

We need to find the right reward function. In order to learn the dynamic scheduling of the MT and LM tasks, we need to assign a reward for using a specific task. Suppose that we sample a task $\tau \in \{\mathrm{MT}, \mathrm{LM}\}$; we then obtain a corresponding batch B_τ and perform gradient descent updating the model parameters from Θ to Θ'. The specific choice of τ has therefore resulted in a parameter change, *and we need to measure how useful it was*. After bench-marking different utility functions, Kreutzer et al. (2021) recommends to measure the loss reduction $L(\Theta) - L(\Theta')$ on a *trusted validation set*. However, while in the setup of Kreutzer et al. (2021) there is a clear choice of the validation set, we are interested in pre-training of an LLM that is then applied to down-stream tasks using the in-context learning paradigm. Therefore, it is not trivial to build a validation set representative of all the possible few-shot tasks. In particular, mitigation strategies would be needed to avoid overfitting to a specific selection of tasks.

We use an intrinsic reward function. In early experiments we contrasted the rewards assigned by each downstream task (e.g. Question Answering) with those assigned by the training tasks and found that the signal from the former was smaller in magnitude and had a bigger variance. *We therefore propose to measure rewards intrinsically on the (pre)-training data itself*. Formally, after taking a gradient step on B_τ, we sample with equal probability a reward task $\rho \in \{\mathrm{MT}, \mathrm{LM}\}$ and obtain a new batch B_ρ on which we measure the loss reduction. We assign equal probability to each reward task as we do not want to fix a preference of one task over the other. *One clear benefit of using an intrinsic reward function is that it is no longer necessary to construct a validation dataset*. While the usage of the training tasks themselves has been considered in (Graves et al., 2017; Kreutzer et al., 2021), they measure rewards on the *same batch B_τ used for taking a gradient step*, while we sample an independent batch B_ρ, possibly from another task. As ρ is sampled with 50% probability to be equal to τ and with 50% probability to be equal to the other task, we *measure both task-specific learning and cross-task transfer*.

The loss reduction needs to be rescaled. Note that the loss scales for LM and MT can be different during training, and so the absolute loss decrease $L(\Theta) - L(\Theta')$ is affected by the task used to compute L. Indeed, in Machine Translation all information content is given in the source sequence and therefore the perplexity of a translation task is generally lower than that of a language modeling task. We solve this problem by computing the reward as *the relative loss reduction*

$$\mathrm{reward} = 1 - \frac{\mathrm{L}(\Theta', \mathrm{B}_\rho)}{\mathrm{L}(\Theta, \mathrm{B}_\rho)}, \tag{3}$$

which was called "pgnorm" in (Kreutzer et al., 2021).

Classical bandit algorithms tend to sample from a single task. The policy from sampling from the two tasks is then learned using multi-armed bandits (Lattimore and Szepesvári, 2020). We initially experimented with EXP3 as in (Graves et al., 2017; Kreutzer et al., 2021). We discovered, however, that the LM task always produces slightly greater reward than the MT task. As EXP3 is designed to pick the best single arm in hindsight, it

Model size (B)	Data Selection	TyDiQA En	TyDiQA Non-En	TyDiQA	XTyDiQA
1.2	LM (100%)	40.23	23.76	25.59	10.40
1.2	LM (90%) – MT (10%)	39.77	25.03	26.67	11.07
1.2	LM (50%) – MT (50%)	41.59	29.42	30.78	13.75
1.2	WARMUP	39.31	23.66	25.40	12.71
1.2	EXP3	**42.50**	30.00	31.39	16.54
1.2	FAIR	41.14	**31.08**	**32.19**	**18.85**
3.8	LM (100%)	47.72	32.97	34.61	13.96
3.8	EXP3	**50.23**	**42.54**	**43.39**	25.82
3.8	FAIR	47.50	36.65	37.85	**26.16**

Table 3: Performance on TyDiQA and XTyDiQA, measured with EM. Static data selection strategies are outperformed by our automated curriculum. Adding parallel data does not hurt performance on En and significantly improves closed generation performance on other languages and (cross-lingual) open generation.

Algorithm 1 FAIR

Require: exploration rate γ, moving average rate μ, number of arms n

1: Initialize arm weights: $w_a \leftarrow 10^{-7}$
2: Compute policy: $\pi_a = (1-\gamma)\frac{w_a}{\sum_a w_a} + \frac{\gamma}{n}$
3: Sample arm: $a \sim \pi$ and get reward r_a
4: Update weights: $w_a \leftarrow (1-\mu)w_a + \mu r_a$

tends to center the policy on the LM arm. To mitigate this issue, we propose a "FAIR" algorithm that samples proportionally to a moving average of the rewards for a given arm, see Algorithm 1 for details. For reproducibility, we provide full details on our curriculum setup in the Appendix.

5 Experimental results

5.1 Baselines

The first baseline we consider is training on just the LM data (LM (100%)); as a grid search on a static mixing ratio λ between the LM and MT tasks is prohibitively expensive, we consider the two values of λ from (Kale et al., 2021): $\lambda = 0.5$ (LM (50%) – MT (50%)) and $\lambda = 0.1$ (LM (90%) – MT (10%)). To create an intermediate behavior between $\lambda = 0.5$, which samples the MT objective more aggressively, and $\lambda = 0.1$, which samples it more conservatively, we consider a WARMUP heuristic that uses $\lambda = 0.4$ for the first 20k steps and then defaults to $\lambda = 0.1$. The value $\lambda = 0.4$ was chosen by inspecting the rewards of each task at the beginning of training.

At model size 1.2B we found that adding parallel data improves performance across the evaluated tasks; however, the automated curriculum learning strategies outperform the other baselines; *thus, given the limited experimental budget, at model size 3.8B we just consider the LM (100%) as a baseline*.

5.2 Question Answering

Our results for Question Answering are in Table 3. *For TyDiQA we see that adding parallel data can significantly improve performance on the non-English part and does not degrade the performance on English*. On XTyDiQA, we observe that adding parallel data can make a significant difference with up to +8 EM points at model size 1.2 billion and +12 EM points at 3.8 billion parameters. Therefore, we see that *including cross-lingual supervision during pre-training improves the open generation abilities of the resulting pre-trained models* for the Question Answering task. We also see that our automated curriculum (either EXP3 or FAIR) outperforms all the other data-sampling strategies at model size 1.2B, and therefore we just experiment with EXP3 and FAIR at the larger model size 3.8B.

5.3 Summarization

Table 4 shows the key outcomes for the summarization task on Wikilingua. We contrasted automated curriculum-based techniques to manual mixing methods. Compared to Question Answering, summarization results sway, albeit not significantly, towards larger models where automated curriculum methods outperform the vanilla LM only method and our proposed FAIR method outperforms the others. Interestingly, we did not observe any gains when scaling from 1.2 billion to 3.8 billion parameters for LM (100%), where the automated curriculum methods benefits from scaling model size more than the than vanilla LM only method.

Model size (B)	Data Selection	En	Non-En	All
1.2	LM (100%)	**16.11**	12.37	12.71
1.2	LM (90%) – MT (10%)	15.99	**12.64**	**12.94**
1.2	LM (50%) – MT (50%)	14.92	11.76	12.04
1.2	WARMUP	15.21	11.82	12.13
1.2	EXP3	15.80	12.25	12.57
1.2	FAIR	14.45	11.55	11.82
3.8	LM (100%)	16.2	12.32	12.67
3.8	EXP3	17.08	13.38	13.72
3.8	FAIR	**18.15**	**14.15**	**14.51**

Table 4: Summarization performance evaluated with RougeL on Wikilingua. At 1.2B parameters, adding more parallel data can slightly decrease performance. However, at 3.8B parameters, adding parallel data slightly improves over the LM-only baseline.

Model size (B)	Data Selection	En → High	High → En	En → Low	Low → En
1.2	LM (100%)	8.96	15.76	0.65	3.01
1.2	LM (90%) – MT (10%)	12.00	20.80	2.00	5.99
1.2	LM (50%) – MT (50%)	17.74	27.14	5.22	13.94
1.2	WARMUP	10.71	21.80	1.28	6.46
1.2	EXP3	16.05	26.20	4.63	18.07
1.2	FAIR	**23.19**	**31.81**	**15.38**	**26.73**
3.8	LM (100%)	12.43	21.02	1.08	5.08
3.8	EXP3	26.63	34.88	23.63	31.53
3.8	FAIR	**30.48**	**36.63**	**27.53**	**36.05**

Table 5: Performance on MT tasks (Flores) measured with sacreBLEU. Adding parallel data greatly improves translation results (evaluated with in-context learning), with the FAIR bandit being the best data selection strategy.

5.4 Machine Translation

Curriculum learning boosts performance. For Machine Translation (Table 5), we partition our analysis into four settings, into- and out-of-English translation (X→En, En→X), and high and low resource translation. *We observe that the gains of using an automated curriculum method can be quite substantial*, with significant gains, e.g. +10 BLEU points, over the LM (50%) – MT (50%) sampling in the En → Low setting. Compared to other methods, our proposed FAIR algorithm also boosts the generation quality further. Note that, given our constrained experimental budget we thus only considered automated curriculum strategies at model size 3.8B.

Translating with control tokens. Recall that parallel data was used in a supervised fashion with the MT objective. For each language, a special control token was prefixed to the source sentence (1). Such language control tokens do not appear in the LM training data; therefore a natural question is whether supplying data to the pre-trained model in the form (1) results in translations to the desired language. This is indeed the case: *at inference time the pre-trained LM performs the supervised task corresponding to each language control token*.

Better translations are generated with in-context learning than by using control tokens. A natural question is whether the resulting models produce better translations with in-context learning or by using control tokens. In Table 6 we compare the "Translation Mode" with control tokens (C) to the one with in-context learning. For in-context learning we use the one-shot setup (O), see the Appendix for examples of how the task is formulated. We clearly see that *the one-shot setup outperforms the one with control tokens, except in the En → Low setting, in which the second is to be preferred*.

Comparison to MTM and LLM baselines. We compare our results to those reported in (Lin et al., 2021), using their same spBLEU implementation to make the comparison fair. The first baseline, M2M (Fan et al., 2021) is an MTM trained with cross-lingual supervision. The other two baselines, XGLM (Lin et al., 2021) and GPT-3 (Brown et al., 2020) consist of LLMs which are trained on a self-supervised objective without any extra crosslingual supervision, and translations are obtained using in-context learning in the few-shot setup.

We do a comparison (Table 7) considering those language pairs for which we also had parallel data. *A clear advantage of our models is that we can al-*

Model size (B)	Data Selection	Translation Mode	En → High	High → En	En → Low	Low → En
1.2	LM (100%)	O	8.96	15.76	0.65	3.01
1.2	LM (90%) – MT (10%)	C	6.66	9.62	2.54	4.12
1.2	LM (90%) – MT (10%)	O	12.00	20.80	2.00	5.99
1.2	LM (50%) – MT (50%)	C	14.68	14.98	15.09	10.31
1.2	LM (50%) – MT (50%)	O	17.74	27.14	5.22	13.94
1.2	EXP3	C	15.23	15.76	13.00	9.52
1.2	EXP3	O	16.05	26.20	4.63	18.07
1.2	FAIR	C	22.44	19.81	34.18	16.84
1.2	FAIR	O	23.19	31.81	15.38	26.73
3.8	LM (100%)	O	12.43	21.02	1.08	5.08
3.8	EXP3	C	19.89	21.50	34.01	18.63
3.8	EXP3	O	26.63	34.88	23.63	31.53
3.8	FAIR	C	21.13	15.95	32.82	16.20
3.8	FAIR	O	30.48	36.63	27.53	36.05

Table 6: Comparison of Translation with control tokens (C) vs the one-shot setup (O) on MT tasks (Flores) measured with sacreBLEU. The one-shot setup yields better results except in the En → Low setting in which using control tokens is better.

ways take the best score between translations generated with control tokens and in-context learning (one-shot): except for En→Fi, we outperform all the other models both at 1.2B and 3.8B parameters scale.

We also looked at those language pairs reported in (Lin et al., 2021) for which we did not have parallel data. For the languages My and Ta our LM dataset had very little data and our models are unable to translate. For Ca, Bg and Hi we found that our systems do generate translations; *when the target language is English, performance can be quite good as we outperform the other systems on the directions Bg→En, Hi→En and Zh→En*. Details of this evaluation are reported in the Appendix.

5.5 Are the gains due to using more multilingual data?

Given that about 77% of the data from (Chowdhery et al., 2022) is in English, *a natural conjecture is that adding parallel data is beneficial because it increases the non-English fraction of the data*. To test this hypothesis we construct a new data-set by taking the non-English side of the MT data and applying to it the LM objective. We still use automated curriculum learning to balance the two fractions of the LM data. In Table 8 we compare the two approaches and *observe a significant performance drop on both Question Answering and Machine Translation when using the MT data with the LM objective*. We conjecture that our MT data might be less useful at modeling language compared to the more rich kind of data from (Chowdhery et al., 2022).

6 Conclusions

We have demonstrated that, when *pre-training Encoder-Decoder large language models*, the inclusion of cross-lingual supervision in the training objective is beneficial. In particular, we have found substantial gains when evaluating the resulting models on Machine Translation and Question Answering. One drawback of including parallel data is the introduction of a new hyper-parameter which quantifies the percentange of such data to use. Even though *inclusion of some cross-lingual supervision is beneficial, determining the optimal amount by a grid search is unfeasible*; however, we have demonstrated that one can get good results by employing automated curriculum learning with multi-armed bandits (Graves et al., 2017). Moreover, in our proposed approach the learned percentage can adjust during training and outperform the static data sampling baselines of (Kale et al., 2021).

7 Limitations

Because of computing limitations, we investigated only Encoder-Decoder models. Further experiments are needed to extend the findings to Decoder-only models. Our summarization evaluations indicate improvements with increasing the parameter count, so further experiments with larger models

Model	Translation Mode	Ar→En	De→En	En→Ar	En→De	En→Fi	En→Fr	En→Ko
M2M-124 (0.6B)	-	25.5	35.8	17.9	32.6	**24.2**	42.0	18.5
GPT-3 (6.7B)	-	10.5	40.4	1.1	25.9	10.2	36.1	1.2
XGLM (7.5B)	-	27.7	38.8	11.5	27.6	23.3	36.0	12.0
Ours (1.2B)	C	20.2	32.9	18.6	34.4	18.7	45.4	10.7
Ours (1.2B)	O	35.0	44.2	19.1	28.7	13.2	40.1	14.5
Ours (3.8B)	C	15.1	24.6	20.5	27.4	19.4	34.7	10.8
Ours (3.8B)	O	**41.2**	**46.9**	**25.5**	**39.0**	21.2	**49.9**	**22.9**
Model	Translation Mode	En→Ru	En→Sw	Fi→En	Fr→En	Ko→En	Ru→En	Sw→En
M2M-124 (0.6B)	-	27.1	26.9	27.2	37.2	20.9	27.5	30.4
GPT-3 (6.7B)	-	11.2	0.5	25.3	42.8	8.3	28.1	5.0
XGLM (7.5B)	-	24.2	18.0	29.2	40.4	19.9	30.4	31.6
Ours (1.2B)	C	23.4	29.8	16.9	35.5	10.7	22.8	16.2
Ours (1.2B)	O	24.1	16.7	27.8	45.5	27.6	35.0	28.9
Ours (3.8B)	C	20.6	**34.7**	18.6	25.6	10.2	20.1	15.6
Ours (3.8B)	O	**31.6**	26.7	**35.5**	**47.6**	**33.1**	**37.5**	**40.3**

Table 7: Comparison to MT baselines, for language pairs for which we included parallel data during training (Flores / spmBLEU). For each language pair, selection of the best strategy between control tokens (C) and in-context learning (O) allows to outperform the baselines.

Data Sel.	Parallel as	TyDi QA	XTyDi QA	En → High	High → En	En → Low	Low → En	Wikilingua
LM	–	25.59	10.40	8.96	15.76	0.65	3.10	12.71
EXP3	MT	31.39	16.54	16.05	26.20	4.63	18.07	12.57
EXP3	LM	29.58	12.67	10.34	16.81	1.22	4.07	11.93
FAIR	MT	32.19	18.85	23.19	31.81	15.38	26.73	11.82
FAIR	LM	18.06	8.53	8.21	15.15	1.89	5.73	10.35

Table 8: Ablation (at model size 1.2B) of using the parallel data with the LM objective. For each data selection strategy, using the non-English size of the parallel data with the LM objective significantly reduces performance on (X)TyDi QA and Translation.

(say $> 8B$ parameters) might be needed to quantify the gains of adding parallel data during pre-training more precisely. Finally, while automated curriculum learning outperformed simple static data sampling strategies, more sophisticated sampling approaches might yield better results.

References

Peter Auer, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E. Schapire. 2002. The nonstochastic multiarmed bandit problem. *SIAM Journal on Computing*, 32(1):48–77.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.

Zewen Chi, Li Dong, Shuming Ma, Shaohan Huang, Saksham Singhal, Xian-Ling Mao, Heyan Huang, Xia Song, and Furu Wei. 2021. mT6: Multilingual pretrained text-to-text transformer with translation pairs. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 1671–1683, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. Palm: Scaling language modeling with pathways.

Jonathan H. Clark, Eunsol Choi, Michael Collins, Dan Garrette, Tom Kwiatkowski, Vitaly Nikolaev, and Jennimaria Palomaki. 2020. TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages. *Transactions of the Association for Computational Linguistics*, 8:454–470.

Angela Fan, Shruti Bhosale, Holger Schwenk, Zhiyi Ma, Ahmed El-Kishky, Siddharth Goyal, Mandeep Baines, Onur Celebi, Guillaume Wenzek, Vishrav Chaudhary, Naman Goyal, Tom Birch, Vitaliy Liptchinsky, Sergey Edunov, Michael Auli, and Armand Joulin. 2021. Beyond english-centric multilingual machine translation. *Journal of Machine Learning Research*, 22(107):1–48.

Sebastian Gehrmann, Abhik Bhattacharjee, Abinaya Mahendiran, Alex Wang, Alexandros Papangelis, Aman Madaan, Angelina McMillan-Major, Anna Shvets, Ashish Upadhyay, Bingsheng Yao, Bryan Wilie, Chandra Bhagavatula, Chaobin You, Craig Thomson, Cristina Garbacea, Dakuo Wang, Daniel Deutsch, Deyi Xiong, Di Jin, Dimitra Gkatzia, Dragomir Radev, Elizabeth Clark, Esin Durmus, Faisal Ladhak, Filip Ginter, Genta Indra Winata, Hendrik Strobelt, Hiroaki Hayashi, Jekaterina Novikova, Jenna Kanerva, Jenny Chim, Jiawei Zhou, Jordan Clive, Joshua Maynez, João Sedoc, Juraj Juraska, Kaustubh D. Dhole, Khyathi Raghavi Chandu, Laura Perez-Beltrachini, Leonardo F. R. Ribeiro, Lewis Tunstall, Li Zhang, Mahima Pushkarna, Mathias Creutz, Michael White, Mihir Sanjay Kale, Moussa Kamal Eddine, Nico Daheim, Nishant Subramani, Ondrej Dusek, Paul Pu Liang, Pawan Sasanka Ammanamanchi, Qi Zhu, Ratish Puduppully, Reno Kriz, Rifat Shahriyar, Ronald Cardenas, Saad Mahamood, Salomey Osei, Samuel Cahyawijaya, Sanja Stajner, Sébastien Montella, Shailza Jolly, Simon Mille, Tahmid Hasan, Tianhao Shen, Tosin P. AMahidewumi, Vikas Raunak, Vipul Raheja, Vitaly Nikolaev, Vivian Tsai, Yacine Jernite, Ying Xu, Yisi Sang, Yixin Liu, and Yufang Hou. 2022. Gemv2: Multilingual nlg benchmarking in a single line of code. *CoRR*, abs/2206.11249.

Alex Graves, Marc G. Bellemare, Jacob Menick, Rémi Munos, and Koray Kavukcuoglu. 2017. Automated curriculum learning for neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1311–1320. PMLR.

Francisco Guzmán, Peng-Jen Chen, Myle Ott, Juan Pino, Guillaume Lample, Philipp Koehn, Vishrav Chaudhary, and Marc'Aurelio Ranzato. 2019. The FLORES evaluation datasets for low-resource machine translation: Nepali–English and Sinhala–English. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 6098–6111, Hong Kong, China. Association for Computational Linguistics.

Mihir Kale, Aditya Siddhant, Rami Al-Rfou, Linting Xue, Noah Constant, and Melvin Johnson. 2021. nmT5 - is parallel data still relevant for pre-training massively multilingual language models? In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 683–691, Online. Association for Computational Linguistics.

Julia Kreutzer, David Vilar, and Artem Sokolov. 2021. Bandits don't follow rules: Balancing multi-facet machine translation with multi-armed bandits. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 3190–3204, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Faisal Ladhak, Esin Durmus, Claire Cardie, and Kathleen McKeown. 2020. WikiLingua: A new benchmark dataset for cross-lingual abstractive summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4034–4048, Online. Association for Computational Linguistics.

Tor Lattimore and Csaba Szepesvári. 2020. *Bandit Algorithms*. Cambridge University Press.

Xi Victoria Lin, Todor Mihaylov, Mikel Artetxe, Tianlu Wang, Shuohui Chen, Daniel Simig, Myle Ott, Naman Goyal, Shruti Bhosale, Jingfei Du, Ramakanth Pasunuru, Sam Shleifer, Punit Singh Koura, Vishrav Chaudhary, Brian O'Horo, Jeff Wang, Luke Zettlemoyer, Zornitsa Kozareva, Mona Diab, Veselin Stoyanov, and Xian Li. 2021. Few-shot Learning with Multilingual Language Models. *arXiv e-prints (to appear in EMNLP)*, page arXiv:2112.10668.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2019. Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv e-prints*.

Machel Reid and Mikel Artetxe. 2022. PARADISE: Exploiting parallel data for multilingual sequence-to-sequence pretraining. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 800–810, Seattle, United States. Association for Computational Linguistics.

Adam Roberts, Hyung Won Chung, Anselm Levskaya, Gaurav Mishra, James Bradbury, Daniel Andor, Sharan Narang, Brian Lester, Colin Gaffney, Afroz Mohiuddin, Curtis Hawthorne, Aitor Lewkowycz, Alex Salcianu, Marc van Zee, Jacob Austin, Sebastian Goodman, Livio Baldini Soares, Haitang Hu, Sasha Tsvyashchenko, Aakanksha Chowdhery, Jasmijn Bastings, Jannis Bulian, Xavier Garcia, Jianmo Ni, Andrew Chen, Kathleen Kenealy, Jonathan H. Clark, Stephan Lee, Dan Garrette, James Lee-Thorp, Colin Raffel, Noam Shazeer, Marvin Ritter, Maarten Bosma, Alexandre Passos, Jeremy Maitin-Shepard, Noah Fiedel, Mark Omernick, Brennan Saeta, Ryan Sepassi, Alexander Spiridonov, Joshua Newlan, and Andrea Gesmundo. 2022. Scaling up models and data with `t5x` and `seqio`. *arXiv preprint arXiv:2203.17189*.

Yi Tay, Mostafa Dehghani, Vinh Q. Tran, Xavier Garcia, Jason Wei, Xuezhi Wang, Hyung Won Chung, Dara Bahri, Tal Schuster, Huaixiu Steven Zheng, Denny Zhou, Neil Houlsby, and Donald Metzler. 2022. Ul2: Unifying language learning paradigms.

David Vilar, Markus Freitag, Colin Cherry, Jiaming Luo, Viresh Ratnakar, and George Foster. 2022. Prompting PaLM for Translation: Assessing Strategies and Performance. *arXiv e-prints*, page arXiv:2211.09102.

Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 483–498, Online. Association for Computational Linguistics.

A Training Hyper-parameters

We train the 1.2B models for 250k steps and the 3.8B models for 500k steps. We use T5X and SeqIO; the input sequences use packing with slightly over 500k non-padding tokens for each batch. The learning rate uses square-root decay, with a base learning rate of 1.0 and 10k warm-up steps. We use the default Adafactor optimizer of the T5X library.

B Technical details for using Automated Curriculum Learning

Here we report a couple of crucial technical details for correctly using Curriculum Learning.

(Graves et al., 2017; Kreutzer et al., 2021) use rescaled rewards:

1. Keep a priority queue of the last T rewards. While the queue is not $x\%$ full, return 0 as a rescaled reward (so the bandit algorithm is not learning).

2. When the queue is $x\%$ full compute the 20-th and 80-th quantiles.

3. When a new reward r comes is, clip it to lie between the current 20-th and 80-th quantiles to obtain r'; then linearly rescale r' to $r'' \in [-1, 1]$ where -1 corresponds to the 20-th quantile and $+1$ to the 80-th quantiles.

4. Supply r'' to the bandit algorithm.

5. Enqueue r and recompute the 20-th and 80-th quantiles.

Therefore, (Graves et al., 2017; Kreutzer et al., 2021) use effective rewards in $[-1, 1]$; however the proofs of convergence for EXP3 in (Auer et al., 2002; Lattimore and Szepesvári, 2020) do not work with negative rewards. Also FAIR can produce negative probability weights if rewards are negative. We therefore rescale rewards so that $r'' \in [0, 1]$ and 0 corresponds to the 20-th quantile of the queue. We use a queue of length $T = 5000$ and $x = 10\%$.

Note also that (Graves et al., 2017) claims to use EXP3S; however, with their choice of parameters it always defaults to EXP3. We therefore do not mention EXP3S in this work as it might be confusing.

C Hyper-parameters for Curriculum Learning

For EXP3 we set the learning rate to 10^{-3} and the exploration rate to 25%.

For FAIR the exploration rate is set to 10% and μ is set to 10^{-2}. Note that μ operates on updating the moving average, so our choice corresponds to a time horizon of 100 steps.

D One-shot task format

Our models are evaluated with in-context learning, using the one-shot paradigm. Here are examples of the formulation for the different tasks.

For Question Answering the input is of the form: "**Context: The European jackal ...\n\n Q: How many jackals ... ?\n A: 70,000\n\n** Context: The first known specimens of ... \n\n Q: When was the ... ? \n A:". The bold part is the single example supplied to use the in-context learning paradigm (here it's one-shot).

For Machine Translation the input is of the form: "**German: Am 28. Juni wurde Marshall Italo Balbo, ...\n English: On June 28, Marshal Italo Balbo, ...\n \n** German: Dr. Ehud Ur, Professor für Medizin ... \n \n English:". The bold part is the single example supplied to use the in-context learning paradigm (here it's one-shot).

For Summarization the input is of the from: "*I will first show a set of step-by-step instructions and then write a short summary of every step in the same language of the instructions.* \n \n **Summarize the following instructions: Loneliness can take ... \n Summary: Identify your type of loneliness. Realize that loneliness is a feeling. Consider your personality. Recognize that you are not alone in feeling lonely. \n \n** Summarize the following instructions: Usually, rainbows are ... \n Summary:". The bold part is the single example supplied to use the in-context learning paradigm (here it's one-shot). The part in italics is a prompt used by the GEM benchmark.

E Translating with Control Tokens

Our models translate sentences to the desired languages when the special language tokens are prefixed to the inputs. In this case the inputs have the following form: "<2en> Dr. Ehud Ur, Professor für Medizin ...", where "<2en>" denotes the task of translating to English.

F Comparison to MTM and LLM baselines for languages without parallel data

We report the comparison to MTM and LLM baselines for those languages for which we did not have

Figure 1: Measuring transfer as the reward on task Y resulting from a gradient step on task X (X to Y in the legend). At the beginning of training the rewards of MT to itself is lower and increases over time. There is always a substantial transfer from MT to LM. We use a running average with window 500 to reduce variance.

parallel data during training in Table 9.

Note that our systems cannot be evaluated with language control tokens when the target language was absent from the parallel data supplied at training time; therefore we evaluate in the one-shot setup. Our systems struggle with the languages My and Ta which were extremely unrepresented in the LM data. For other languages, we have indication that the model is able to translate. Moreover, performance on translation to English can be quite good, e.g. outperforming the other systems especially on the pairs Bg→En, Hi→En and Zh→En.

Note also that the model trained with FAIR outperforms the model trained with LM data only. Therefore, parallel data has improved translations also to language pairs that were not present in the cross-lingual supervised data.

G Visualizing the rewards during training

We plot in Figure 1 the transfer between tasks, measured in terms of rewards. Specifically, when we took a gradient step on task X and evaluated the reward on task Y we get a measure of transfer from X to Y that can be plotted over time. Note that there is always positive transfer from the MT to the LM objective; however the LM objective has on average higher rewards when applied to MT or LM. Interestingly, the transfer from MT to itself was low at the beginning of training and increased over time.

Model	Bg→En	Ca→En	En→Bg	En→Ca	En→Hi	En→My
M2M-124 (0.6B)	33.0	33.4	**37.4**	31.2	**28.1**	3.5
GPT-3 (6.7B)	21.6	40.2	5.9	23.8	0.3	0.1
XGLM (7.5B)	35.5	**41.1**	33.1	**34.0**	19.9	**11.0**
Ours (FAIR, 3.8B)	**36.9**	39.4	16.8	19.3	8.0	0.4
Ours (100% LM, 3.8B)	25.3	29.7	12.1	14.9	3.3	0.4
Model	**En→Ta**	**En→Zh**	**Hi→En**	**My→En**	**Ta→En**	**Zh→En**
M2M-124 (0.6B)	3.4	**19.3**	27.9	10.0	8.3	20.9
GPT-3 (6.7B)	0.0	12.5	1.2	0.5	1	21.1
XGLM (7.5B)	**8.5**	15.6	25.2	**14.1**	**16.3**	20.7
Ours (FAIR, 3.8B)	0.4	12.0	**28.6**	2.0	6.4	**25.4**
Ours LM only (100% LM, 3.8B)	0.4	8.0	11.6	2.0	3.4	17.1

Table 9: Comparison to MT baselines, for language pairs for which we did not include parallel data during training (Flores / spmBLEU). In our stystems, adding parallel data improves one-shot translation performance also on language pairs that were not included in parallel data.